



BOOTSTRAP

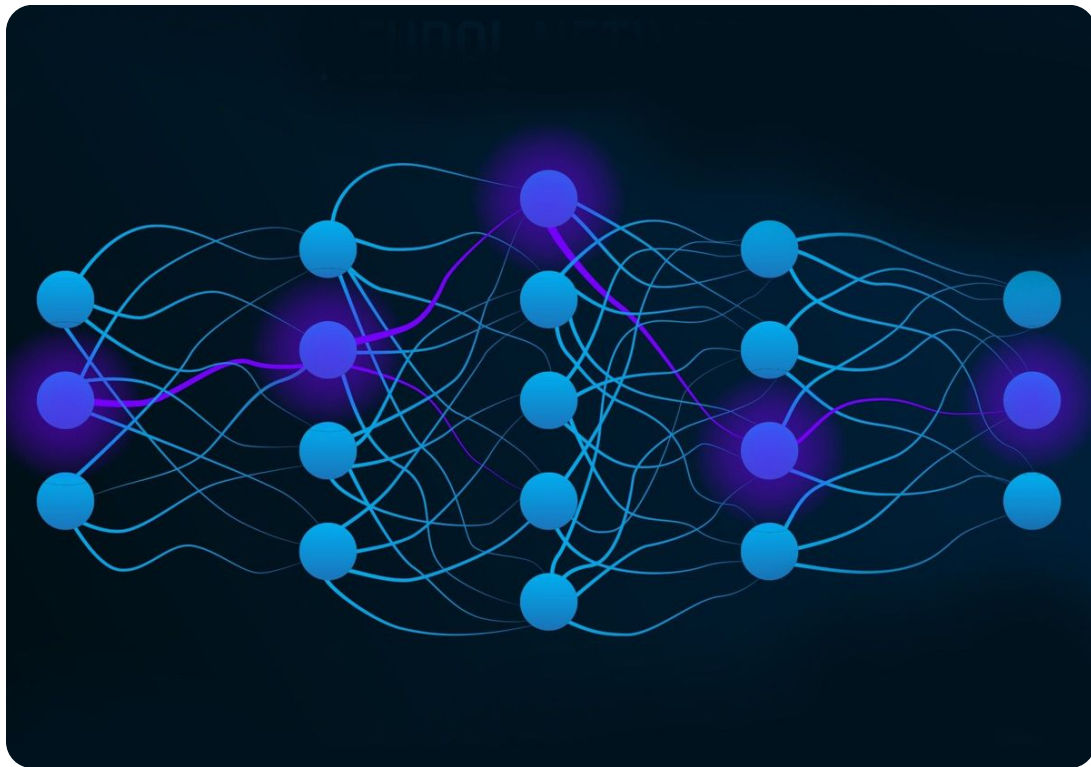
< REINFORCEMENT LEARNING />



BOOTSTRAP

Reinforcement Learning

Reinforcement learning (RL) is a branch of machine learning where agents learn to **make decisions by interacting with an environment** and receiving rewards. It is widely used in domains such as robotics, gaming, and autonomous systems. Over the last few years, breakthroughs in deep learning, high-performance computing, and large-scale simulation have opened up new possibilities for it, such as enabling self-driving cars to navigate complex traffic, training virtual agents to play games at human or superhuman levels, and optimizing energy consumption in smart grids. By exploring RL, you will learn how to model problems as Markov decision processes, design reward structures, and implement algorithms like **Q-learning, policy gradients, and actor-critic methods**. The practical project in this module will allow you to apply these concepts to a real-world scenario, demonstrating the power and versatility of reinforcement learning in solving dynamic decision-making challenges.



Reinforcement Learning

Introduction

Reinforcement learning (RL) is a subset of machine learning concerned with how an intelligent agent should take actions in a dynamic environment in order to maximize a reward signal.

A typical reinforcement learning scenario would be composed of those steps :

- ✓ Agent **observes** the current situation
- ✓ Agent chooses an **action** based on what it sees
- ✓ Environment responds with a **new situation** and a **reward** (game state changes, score updates)
- ✓ **Repeat** until the episode ends

The purpose of reinforcement learning is for the agent to learn an optimal (or near-optimal) policy that maximizes the reward function.



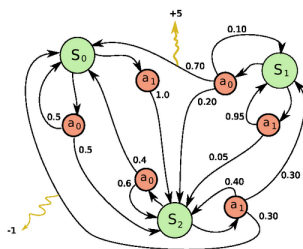
Rewards can be of any value, not only positive

On Paper

Those steps can be written down and modeled after a [Markov's Chain](#), where each node is a situation and changes mean travelling from a node to another. You can assign a reward (positive or negative) to each link.

You'll find a set of states from a game called Hearthstone (specifically the Poisonmaster Pollark's Lethal Puzzle #2). Try and draw links between them and **create a Markov's chain** in the tool of your choice. Let's say that at its initial state, your model has an equal chance of using all options at its disposal.

How would you go about assigning a reward to each action? Each path? Each state?



1	2	3	4	5	
1	-	+1	-	+1	
2	-	+1	-1	+1	
3	-	+1	-1	+1	
4	-	+1	-1	-1	
5	-	+1	+1	-	
...					
23	+1	-	-1	+1	
24	+1	-	-1	-1	
25	+1	-	+1	-	

A Markov's Chain with 3 states, two actions and rewards associated with some of the paths

Another type of visualisation

Solved?

We've created a simple situation with only 4 possible actions (without replacement) yet we already would have had to handle 40 different states, for which we set rewards values by hand. A possible heuristic for doing so might be rewarding the action that deals the most damage. Does this heuristic work every time? What if we had 5 of mana instead of 4? If the wyvern started with 5 atk? If we had a spell that cost 1 and did nothing?



Can you imagine how many states that would be? We might need a computer to compute the rewards now...

Q-Learning

Without a self-evident heuristic, a Markov chain can be a good tool because it allows us to see our problem as a Markov decision process. And those are 🎉 SOLVED 🎉 by Q-learning!

Q-learning (a well-known reinforcement learning algorithm) can “identify an optimal action-selection policy for any given finite Markov decision process, given infinite exploration time and a partly random policy”.



Hmm... We're gonna need a big computer.

Other solutions - Deep Q-Learning

One of the main drawbacks of Q-learning is that it becomes infeasible when dealing with large state spaces, as the size of the Q-table grows exponentially with the number of states and actions. In short, we never have “infinite exploration time”

Most of Reinforcement Learning's skill will be about making the most out of our limited exploration time. This objective will be the driving force behind the domain's innovations:

- Deep Q-Learning and neural nets
- Double DQN
- Dueling DQN
- Prioritized experience replay

It should be the driving force behind your choices and decisions:

- Which parameters do you use?
- What is your exploration policy?
- Do you use vectorized envs?

Gymnasium and Pole Balancing

For this bootstrap, we'll create a reinforcement scenario using **Gymnasium**, an API standard for reinforcement learning with a diverse collection of reference environments. Gymnasium is a maintained fork of OpenAI's Gym library. The code you'll produce will be in **python**.

You can look at Gymnasium's documentation [here](#)

In this scenario we will tackle an inverted-pendulum control task (like balancing a pole on a seal's nose), through an exercise called CartPole-v1.

In CartPole-v1, the **action space is discrete** and the observation is a **4-D state** (cart position/velocity, pole angle/angular velocity). The agent receives **+1 reward per step**; an episode ends if angle or cart position exceed limits (terminated) or when the **500-step time limit** is hit (truncated).



On a computer

For this exercise, you'll need python and gymnasium (and maybe a neural net library)

For each “environment” you want to create scenarios in, you may also have to install dependencies. In this instance:

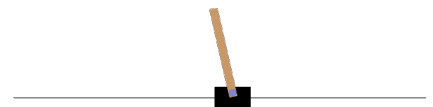
```
Terminal
$> pip install gymnasium[classic-control]
```

Try the example code from the gymnasium's page. You should see a window opening and showing a cart with a pole. The cart moves randomly left and right, and the pole eventually falls over. This is expected - the agent is acting randomly!

More Episodes

The window closing as quickly as it does isn't optimal. We could want to “save” an episode in order to analyse it and build an heuristic based on our observations.

Gymnasium has a video recording functionality for this exact reason. Use it, and generate videos of episodes for a few different seeds that you can look at for reference.



Policy Update

Our “policy” (the code that dictates which action to take depending on the state) is either random or a simple heuristic at this stage. It is also deterministic, and hasn’t yet begun to learn.

In reinforcement learning, there are a lot of different approaches you can take. Here are questions you should ask yourself about your chosen method:

- Is it on-policy or off-policy?
- Does it favor exploration or exploitation?
- How are your rewards distributed?

How do you avoid the credit assignment problem?

Can you keep this pole upright?

Reproducibility

You can make your model start its training from scratch with a new seed. Seed everything (Python, NumPy, Torch, and `env.reset(seed=...)`)! Train your model again and again and log your results. Can you consistently achieve $\pm 95\%$ CI on returns/episode length?

v 0.3

{EPITECH}